Universal adversarial perturbations: what I did, what inspired it, what comes next

Project overview for the group. Reproduction of Moosavi-Dezfooli et al. (CVPR 2017) in PyTorch, extension to six modern classifiers, and a study of a Gaussian-noise defense against a non-adaptive and an adaptive (EOT) attacker. November 2026.

In short.
1. I ported the original code to PyTorch and reproduced the paper on ImageNet: one invisible perturbation (±10/255 per pixel) fools **81.0%** of 40,000 unseen images on ResNet-50, and 97.5% of the fooled images become “spider web”.
2. On six architectures, the perturbation fools **72–83%** of the images. The three attention-based models (MaxViT, Swin, ViT) are harder to fool than the three CNNs, and four of the six models are pushed towards the **same class, “brain coral”**.
3. Adding Gaussian noise ($\sigma = 20/255$) to the input **neutralises** these perturbations (fooling rate 80% → 7–21%) for a few points of clean accuracy.
4. But an attacker who knows the defense and averages over the noise (EOT) **breaks it**: 45–50% fooling at the same budget $\xi = 10/255$, and 86% at $\xi = 16/255$. The noise only forces the attacker to use a larger budget. Under EOT, the dominant class changes and the perturbation becomes coarser.

1. Starting point: universal adversarial perturbations

An adversarial perturbation is a small vector r such that a classifier changes its prediction on x + r. It is usually computed for one image. Moosavi-Dezfooli, Fawzi, Fawzi and Frossard (CVPR 2017) showed that a **single** vector v, computed once, fools a network on most natural images: it is universal. With $\|v\|_\infty \le 10/255$, it fools 78–94% of the ImageNet validation images on CaffeNet, VGG, GoogLeNet and ResNet-152, and it transfers between architectures.

The algorithm. Go through a set X of images. For each image that the current v does not fool yet, compute with DeepFool the smallest step that sends x + v across the nearest decision boundary, add it to v, and project v back onto the ℓ_∞ ball of radius ξ. Repeat passes over X until 80% of X is fooled. DeepFool (Moosavi-Dezfooli, Fawzi, Frossard, CVPR 2016) linearises the classifier around the current point and jumps to the closest linearised boundary, with a 2% overshoot.

2. What I built

- **PyTorch port** of the original TensorFlow code (`universal_pert.py`, `deepfool.py`, `demo.py`), for any torchvision model, with the ImageNet normalisation inside the model, batched evaluation and checkpoints so that a computation killed on a shared machine can resume.
- **A sanity check of DeepFool on a logistic regression** (`testlogistic/`). For an affine classifier the minimal perturbation is known in closed form, and DeepFool must find it in one iteration. It does: 803 of 803 points fooled, 1.00 iteration on average, maximal error $2.9 \cdot 10^{-6}$.
- **Data**: the 50,000 ImageNet validation images (Hugging Face `ILSVRC/imagenet-1k`), split into X = 10 images per class (10,000 images, used to compute v) and 40 images per class for evaluation (40,000 images, never seen by the algorithm). The paper takes X from the training set; here both parts come from the validation set, disjoint. Imagenette (10 ImageNet classes) is used for quick tests on my laptop.
- **Evaluation** (`evaluate.py`): accuracy, fooling rate (fraction of images whose predicted class changes), dominant classes, and a **control**: v with its pixels shuffled, which has exactly the same norms but no structure.

- **Defense and adaptive attack** (`eval_noise.py`, EOT options in `demo.py`), described in sections 5 and 6.

The ImageNet computations ran on the school GPU machines (RTX A4000), the small ones on my laptop.

3. Experiment 1: reproducing the paper

Setting	X	Evaluation	Clean acc.	Acc. with v	Fooling rate	Control
ResNet-18, Imagenette	500 images, 10 classes	3,925 images	77.7%	67.4%	24.0%	8.1%
ResNet-50, ImageNet	10,000 images, 1000 classes	40,000 images	80.5%	17.1%	**81.0%**	9.5%

With the protocol of the paper, the result is reproduced: 81.0% on ResNet-50, within the 78–84% that the paper reports for comparable networks, and almost the same on X (82.3%) as on unseen images, so the perturbation **generalises**. With only 500 images of 10 classes, it does not (85.6% on X, 24.0% on new images): the algorithm reaches 80% on X too early and v "overfits" X. The perturbation fools 8 times more than the shuffled control, so it is its **direction** that matters, not its size.

Dominant class. 97.5% of the fooled images are classified "spider web". On a uniform gray image, v alone is classified "spider web" with probability 0.99: the perturbation is a spider-web texture for the network.

4. Experiment 2: six architectures

Same protocol, six pretrained torchvision models: two classic CNNs (ResNet-18, ResNet-50), a modern CNN (ConvNeXt-B), a hybrid of convolution and attention (MaxViT-T), a windowed Transformer (Swin-B) and a plain Vision Transformer (ViT-B/16).

Model	Family	Clean acc.	Acc. with v	Fooling (eval.)	Fooling on X	Passes	Dominant class
ResNet-18	CNN	68.8%	18.0%	**79.6%**	82.1%	2	brain coral (18%)
ResNet-50	CNN	80.5%	17.1%	**81.0%**	82.3%	4	spider web (97.5%)
ConvNeXt-B	modern CNN	83.9%	15.5%	**82.9%**	83.5%	1	brain coral (97.6%)
MaxViT-T	conv. + attention	83.5%	23.0%	**74.9%**	80.1%	3	brain coral (98.9%)
Swin-B	windowed Transformer	83.3%	23.9%	**73.8%**	80.5%	5	brain coral (97.8%)
ViT-B/16	Transformer	80.8%	25.7%	**71.9%**	72.8%*	9*	scoreboard (96.8%)

* ViT never reached 80% on X: it plateaued around 73% from the 6th pass on, and I stopped it during the 10th pass. The share of the dominant class is among the fooled images.

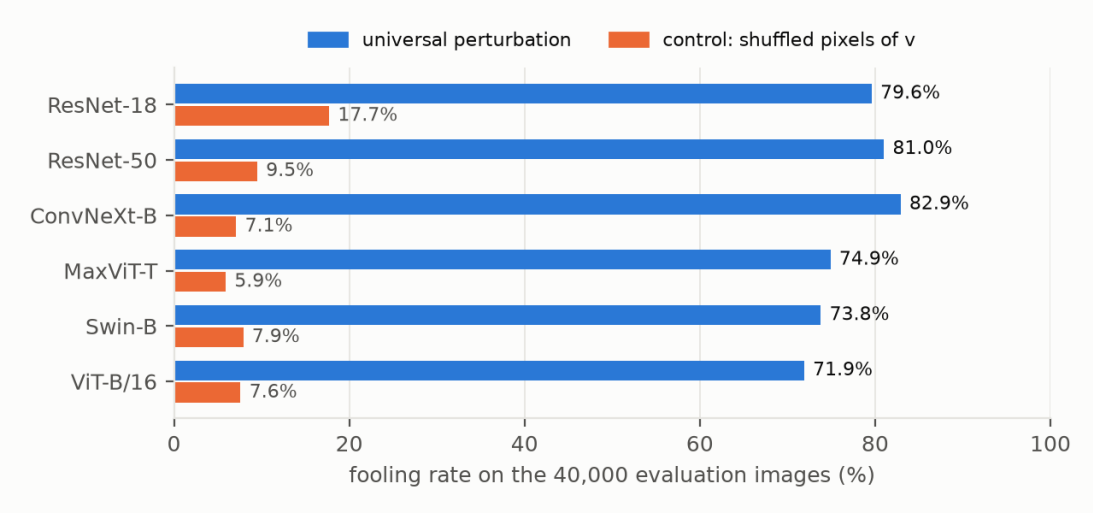


Figure 1. Fooling rate on the 40,000 evaluation images, against the control.

Attention-based models resist better. CNNs: 80–83%; attention-based models: 72–75%. The algorithm needs more passes on them (ConvNeXt: 1, ViT: never), and their perturbation generalises less well from X to new images (MaxViT −5.2 and Swin −6.7 points, against −0.6 to −2.5 for the CNNs). ConvNeXt, the most accurate model, is also the easiest to fool, so robustness does not follow accuracy. Caveat: one model per family and one run per model, and the algorithm stops at a fixed target (80% on X), not at a fixed effort.

Four architectures, one class. Four independently computed perturbations, on very different networks, all send the images to “brain coral”. This suggests that the dominant class comes from the **training data** rather than the architecture: a class that is defined almost only by a texture (tight, winding ridges) that a pattern repeated over the whole image can imitate. ViT is the exception, and its perturbation is visibly a grid of 14 × 14 squares of 16 pixels: exactly its patch grid.

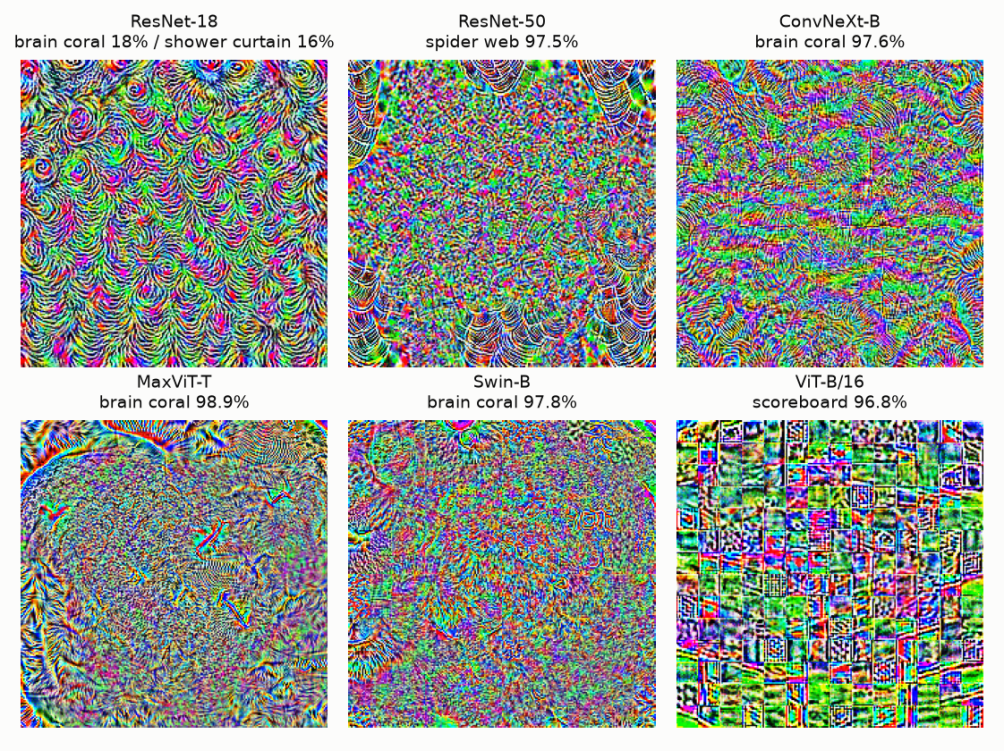


Figure 2. The six perturbations, stretched to [0, 1] to be visible (real values within ±10/255).

5. Experiment 3: a Gaussian-noise defense (non-adaptive attacker)

Idea. DeepFool takes minimal steps: each image is pushed just across the boundary (2% overshoot). The universal perturbation should therefore be fragile. The defense adds independent Gaussian noise of standard deviation σ to every image before the classifier, at test time, without retraining. Here the perturbation is the one computed without knowledge of the defense.

Columns of `eval_noise.py`: A = accuracy on clean images + noise (cost of the defense), B = accuracy on attacked images + noise, C = fooling rate under the noise (prediction on clean + noise vs attacked + noise), P = noise floor (C with v = 0: what two noise draws change on their own).

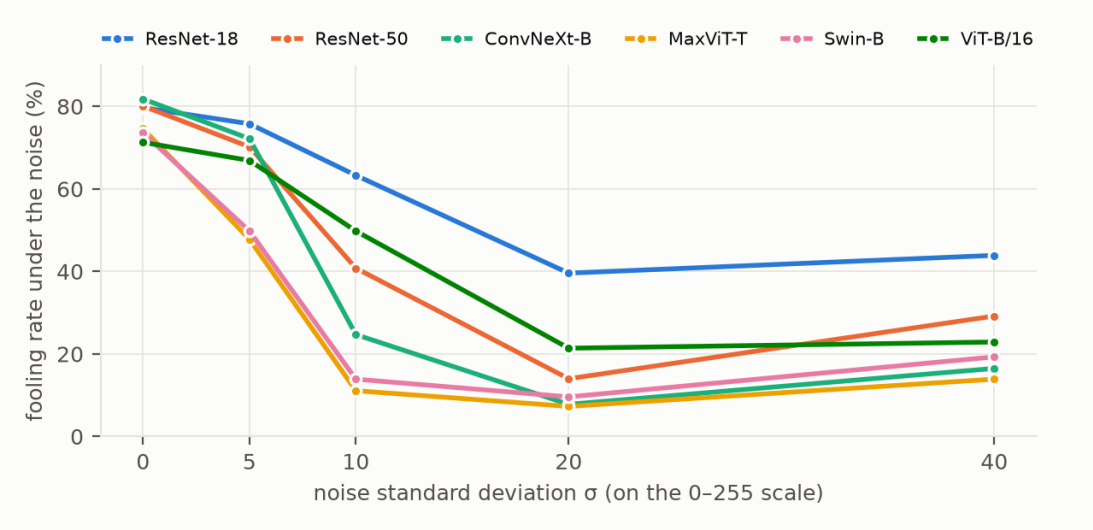


Figure 3. Fooling rate under the noise for the six models (ImageNet, 5,000 evaluation images).

Model	C without defense	C at σ = 10	C at σ = 20	Clean accuracy lost at σ = 20
ResNet-18	79.7%	63.3%	39.6%	−17.3 points
ResNet-50	80.1%	40.8%	14.0%	−9.2 points
ConvNeXt-B	81.8%	24.7%	7.8%	−5.6 points
MaxViT-T	74.6%	**11.1%**	7.3%	−4.2 points
Swin-B	73.6%	**13.9%**	9.6%	−6.0 points
ViT-B/16	71.3%	49.8%	21.4%	−4.6 points

The noise neutralises the perturbation on every model. MaxViT and Swin only need σ = 10 (fooling 11–14% for 1.6–2.4 points of clean accuracy). ViT is the exception: its patch-grid perturbation survives noise best. ResNet-18 is a bad candidate: the noise alone costs it 17 points. Above σ = 20, the noise itself starts to change predictions. The same experiment on Imagenette (ResNet-50, 1,000 images) gives 84.8% → 10.5% at σ = 20 for 4.5 points of clean accuracy.

This does not show that the defense is good. The attacker did not know about it. The standard since Athalye, Carlini and Wagner (2018) is to evaluate a randomised defense against an attacker who knows it and optimises the expectation over the randomness (EOT).

6. Experiment 4: an adaptive attacker (EOT)

EOT (Expectation Over Transformation). The defense draws a new noise for every prediction, so the attacker cannot predict it. Instead, the attacker looks for a perturbation that works on average over the noise: at every step it evaluates several noisy copies of x + v and follows the average gradient.

I implemented two versions. **DeepFool + EOT**: the algorithm of the paper, where an image counts as fooled if most of its noisy copies are, and the DeepFool steps of several noisy copies are averaged. **Gradient + EOT** (`--algo sgd`): signed gradient ascent on the cross-entropy, averaged over a batch of 32 images × 4 noise draws, projected on the ℓ_∞ ball; it keeps pushing images that are already fooled, which gives margin. Optionally **targeted** (`--target`): push every image towards one class.

First attempt (Imagenette, 500 images): inconclusive. All attacks stayed at the noise floor. In hindsight the setting was too small (even without defense, a perturbation computed on 500 images only fools 32.5% of new images) and the attack was under-tuned (fixed step, 10 epochs). Lesson: test the defense where the attack works.

ImageNet (ResNet-50, X = 10,000 images, evaluation on 5,000 others, defense σ = 20/255).

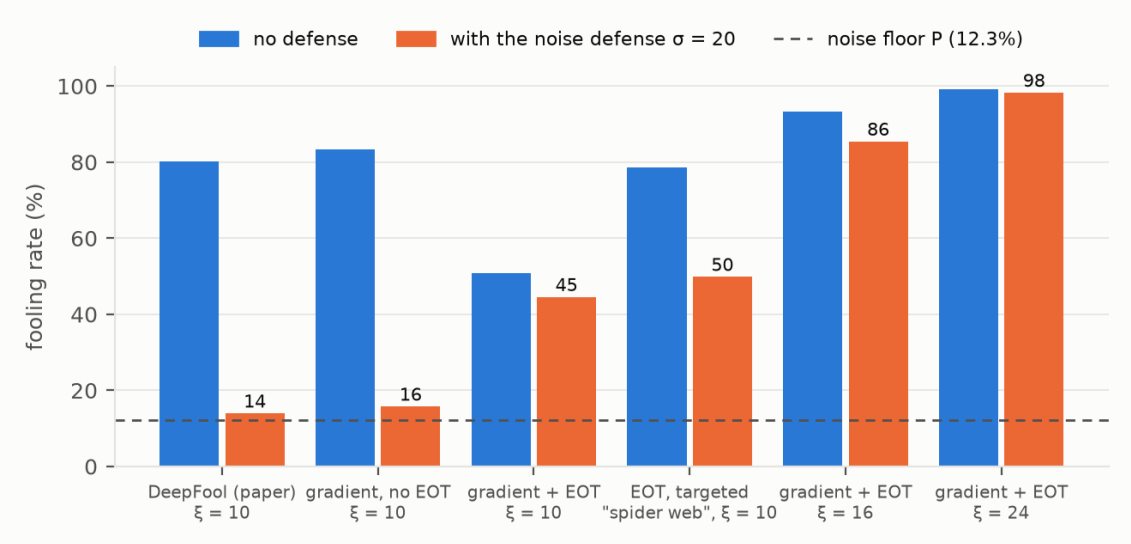


Figure 4. Fooling rate without and with the noise defense. With the defense, the clean accuracy is 71.4% (80.8% without) and the noise alone changes 12.3% of the predictions.

Perturbation	Fooling, no defense	Fooling, σ = 20	Accuracy, σ = 20	Dominant class under the noise
DeepFool (paper), ξ = 10	80.1%	14.0%	70.2%	— (spider web 0.1%)
Gradient, no EOT, ξ = 10	83.4%	15.8%	68.6%	—
Gradient + EOT, ξ = 10	50.8%	**44.6%**	48.9%	none (dispersed)
EOT targeted “spider web”, ξ = 10	78.7%	**49.9%**	42.7%	spider web (38.8% of all images)
Gradient + EOT, ξ = 16	93.3%	**85.5%**	13.0%	guinea pig (67.9%)
Gradient + EOT, ξ = 24	99.2%	**98.3%**	1.4%	dispersed (bluetick without noise)

What this shows.

- **The control isolates EOT.** The gradient perturbation without EOT is as strong as the paper's without defense (83.4%) and just as fragile (15.8%). What breaks the defense is taking the noise into account.
- **The defense is broken.** At the same budget ξ = 10, the adaptive perturbation fools 45–50% of the images despite the noise; at ξ = 16, 86%. As in the literature on randomised defenses, the defense mostly fooled the non-adaptive attacker.
- **Cost/protection for the defender.** The defender pays 9.4 points of clean accuracy. Against an adaptive attacker at ξ = 10 it keeps about 45% accuracy (17% without defense): the defense still helps, but far less than the non-adaptive evaluation suggested (70%). At ξ = 16 it keeps 13%. The noise roughly forces the attacker to multiply its budget by 1.6.

- **Robustness costs nominal strength.** Without defense, the EOT perturbation at ξ = 10 fools less than the normal one (50.8% vs 83%), but it barely loses anything under the noise.
- **The dominant class changes.** Untargeted EOT loses “spider web”; with a larger budget a new class appears (“guinea pig” at ξ = 16). A targeted attack can still impose “spider web” on 39% of the images through the noise. The EOT perturbations are made of larger, more contrasted stripes and fur-like strokes, and saturate 29–41% of the pixels (6% for DeepFool). The targeted one has 31% of its energy at low frequencies, against 7–9% for the others.

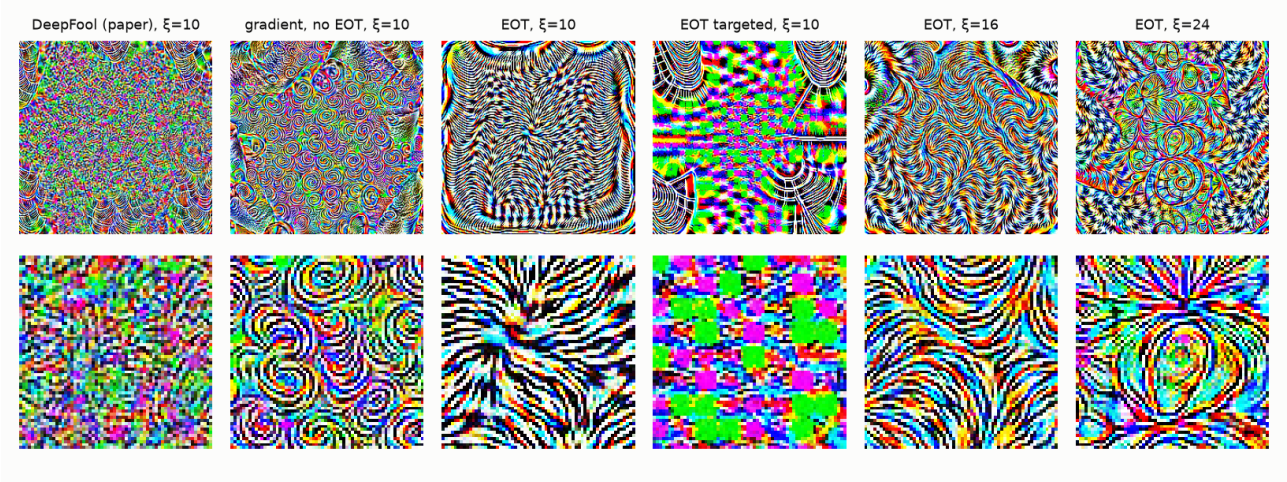


Figure 5. Perturbations of ResNet-50: whole image (top) and central 64 × 64 pixels (bottom). The perturbations computed with EOT are coarser.

Caveats: one model, one noise level, one run per perturbation, about ±1.4 points of sampling error on 5,000 images. “Fur-like” and “coarser” are my reading of the images; the frequency measure is crude.

7. The papers behind each experiment

Paper	What I took from it	Used in
Moosavi-Dezfooli, Fawzi, Fawzi, Frossard. Universal adversarial perturbations. CVPR 2017. arxiv.org/abs/1610.08401	The algorithm, the protocol (ξ = 10/255, X = 10 images per class), the dominant classes	Exp. 1, 2
Moosavi-Dezfooli, Fawzi, Frossard. DeepFool. CVPR 2016. arxiv.org/abs/1511.04599	The minimal-step attack inside the algorithm; tested on a logistic regression	Exp. 1
He et al. ResNet (CVPR 2016); Liu et al. ConvNeXt (CVPR 2022); Tu et al. MaxViT (ECCV 2022); Liu et al. Swin (ICCV 2021); Dosovitskiy et al. ViT (ICLR 2021)	The six architectures (torchvision weights)	Exp. 2
Xie et al. Mitigating adversarial effects through randomization. ICLR 2018. arxiv.org/abs/1711.01991; Guo et al. Countering adversarial images using input transformations. ICLR 2018. arxiv.org/abs/1711.00117	Random input transformations as a defense, which looked very strong against non-adaptive attacks	Exp. 3
Cohen, Rosenfeld, Kolter. Certified adversarial robustness via randomized smoothing. ICML 2019. arxiv.org/abs/1902.02918	Gaussian noise as a principled defense; its certified radii ($\ell_2 \leq 0.5$–3) are far below $\|v\|_2 \approx 10$, so it gives no guarantee here	Exp. 3
Athalye, Carlini, Wagner. Obfuscated gradients give a false sense of security. ICML 2018. arxiv.org/abs/1802.00420; Athalye, Engstrom, Ilyas, Kwok. Synthesizing robust adversarial examples. ICML 2018. arxiv.org/abs/1707.07397	EOT and the rule of evaluating a defense against an adaptive attacker	Exp. 4
Brown et al. Adversarial patch. 2017. arxiv.org/abs/1712.09665	A universal attack that is robust to random transformations: evidence that universal + EOT should work	Exp. 4
Xu, Singh. Robust universal adversarial perturbations. ICML 2024. arxiv.org/abs/2206.10858	Closest work: a gradient + EOT UAP survives transformations of the perturbation, DeepFool + EOT much less. Our ImageNet results agree. They do not measure the defender's clean accuracy nor the dominant classes	Exp. 4
Shafahi et al. Universal adversarial training. AAAI 2020. arxiv.org/abs/1811.11304	The gradient-based UAP algorithm and the clipped loss option	Exp. 4
Gao et al. On the limitations of stochastic pre-processing defenses. NeurIPS 2022. arxiv.org/abs/2206.09491	Why more randomness costs clean accuracy (robustness/invariance trade-off)	Exp. 3, 4

A full literature review (in French, being translated) is in `reports/`. Its main conclusion: I found no paper that evaluates Gaussian noise at inference against a UAP recomputed with EOT on ImageNet, measuring both the defender's clean accuracy and the attack, nor any study of how the dominant class changes under EOT. Confidence in these gaps is moderate: the search did not systematically go through all papers citing Xu & Singh.

8. What I plan to do next

- **The full cost/protection curve.** Repeat the EOT attack for σ = 10 and 40 and several budgets ξ, and plot the defender's accuracy against the attacker's budget. This is the quantitative answer to “is noise worth it?”.
- **The same on a Transformer** (Swin-B or ViT-B/16, `MODEL=swin_b ./run_eot_imagenet.sh`). Transformers were easier to defend against the non-adaptive attack: are they still against EOT?
- **The dominant class under EOT** (the most original question): which classes appear, are they “coarse-texture” classes, and does the perturbation really move to lower frequencies? Tests: gray-image test, spectrum of v, and an EOT attack with a low-pass constraint.
- **Transfer matrix** between the six models: does the “brain coral” perturbation of Swin also send ConvNeXt to “brain coral”?
- **Robustness of the conclusions**: several runs (seeds) for the main numbers, and a comparison of the models at a fixed number of passes rather than at a fixed target.

Reproducing

`python demo.py -m resnet50 -t <X>` computes a perturbation; `evaluate.py` evaluates it; `eval_noise.py` evaluates it under the noise defense; `run_models.sh` runs experiment 2; `run_eot_imagenet.sh` runs experiment 4. All numbers and figures are in `python/results/`. Other documents in `docs/en/`: the analysis of the six models, and a review of random-transformation defenses.